

EXERCISE 17 · UNIT 3

# First CUDA kernels — indexing, guards and transfers

Three small kernels that teach the four things every CUDA program does, and the three mistakes that make a CUDA program wrong without making it crash.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

---

EXERCISE

ex17-cuda-vector-add

LANGUAGE

CUDA C++

SUBMIT AS

<ROLLNO>.cu

UNIT

3 — GPU and CUDA

MARKS

10

OFFERING

Odd Semester 2026-27

# What you are building

Three functions, declared in `include/vecops.h`. Each takes **host** pointers, so each must do the full round trip:

1. `cudaMalloc`            allocate on the device
2. `cudaMemcpy H2D`        copy inputs across
3. `kernel<<<b,t>>>`        launch
4. `cudaMemcpy D2H`        copy the result back
5. `cudaFree`              release

| FUNCTION                                | COMPUTES                                                          |
|-----------------------------------------|-------------------------------------------------------------------|
| <code>gpu_saxpy(a, x, y, out, n)</code> | <code>out[i] = a*x[i] + y[i]</code>                               |
| <code>gpu_elemmax(x, y, out, n)</code>  | <code>out[i] = max(x[i], y[i])</code>                             |
| <code>gpu_count_above(x, n, t)</code>   | how many elements are <b>strictly</b> greater than <code>t</code> |

# The idiom, and the guard

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) { /* work on element i */ }
```

The grid size is computed by **ceiling division**:

```
int blocks = (n + THREADS - 1) / THREADS;
```

so `blocks * THREADS` is almost always larger than `n`. With `n = 1000` and 256 threads per block you launch 1024 threads, and 24 of them have `i >= n`.

## **PITFALL · WITHOUT THE GUARD, THOSE 24 THREADS WRITE PAST THE END OF YOUR ARRAY**

Nothing crashes. No error is reported. Memory that belongs to something else is quietly overwritten, and the bug surfaces somewhere unrelated, later.

One hidden test allocates a guard band past element `n-1`, fills it with a sentinel value, and checks the sentinel survives. A missing bounds guard fails that test and nothing else, which is exactly how this bug behaves in real code.

# The two other silent mistakes

## PITFALL · A KERNEL LAUNCH IS ASYNCHRONOUS

`kernel<<<b,t>>>(...)` returns immediately; the work has not happened yet. `cudaMemcpy` happens to synchronise, which is why the pattern above works. But if you ever time a launch without `cudaDeviceSynchronize()`, you measure the launch, not the computation, and conclude your kernel is impossibly fast.

## PITFALL · LAUNCH ERRORS ARE REPORTED LATE

A bad launch configuration or an out-of-bounds access surfaces at some *later* CUDA call, which makes the error message point at innocent code. Check explicitly:

```
kernel<<<blocks, threads>>>(...);
CUDA_CHECK(cudaGetLastError());    /* launch configuration */
CUDA_CHECK(cudaDeviceSynchronize()); /* errors during execution */
```

`CUDA_CHECK` is provided in the starter. Use it on every CUDA call. It costs nothing and it turns a mystery into a file and line number.

# The counter is a race, again

---

`gpu_count_above` needs every thread to contribute to one number. A plain `(*count)++` from thousands of threads is the same lost-update race you met in OpenMP in Exercise 6, for exactly the same reason: read, modify, write is three operations, not one.

On a GPU the cheap fix is an atomic:

```
if (i < n && x[i] > t) atomicAdd(count, 1ULL);
```

Any correct approach is accepted, including a tree reduction in shared memory if you want to look ahead to Exercise 20. One hidden test runs the count three times over four million elements: an unsynchronised counter usually gets a different wrong answer each time.

## NOTE · STRICTLY GREATER

`gpu_count_above(x, n, t)` counts elements with `x[i] > t`, not `>=`. There is a test for exactly this.

# Building and running

```
./selfcheck.sh          # compiles and runs the public tests via srun
./selfcheck.sh AIE23001.cu # test a specific file
```

By hand:

```
export PATH=/usr/local/cuda/bin:$PATH
nvcc -O2 -std=c++17 -arch=sm_80 -Xcompiler "-Wall,-Wextra,-Werror" \
      -Iinclude -I../harness vecops.cu tests/public.cu -o t -lm
srun --gres=gpu:1 --time=00:10:00 --mem=8G ./t
```

## ON ASAI COMPUTE · WHERE TO RUN IT

You need a GPU, and the login node is not where you get one. `selfcheck.sh` goes through `srun` automatically when SLURM is available. Compiling on the head node is fine; running is not.

# How this is marked

| CHECK                        | MARKS | WHAT IT MEANS                                                               |
|------------------------------|-------|-----------------------------------------------------------------------------|
| Compiles cleanly             | 1     | Builds with warnings as errors                                              |
| Index idiom and bounds guard | 2     | The standard expression and an <code>if (i &lt; n)</code> guard are present |
| Public tests                 | 3     | The tests shipped with the exercise                                         |
| Hidden tests                 | 4     | Edge cases, a guard-band overrun check, and a repeated race check           |

# Submitting

---

```
AIE23001.cu
```

One file. No `main()`, no archive.